EEMS

Frontend Development Blueprint

Angular UI/UX | Screen inventory | Dynamic navigation | Typed API integration

IMPLEMENTATION-READY DESIGN BASELINE | VERSION 1.0

Audience: Solution architects, engineering leads, developers, QA, DevOps and project owners.

Scope: Multi-tenant election operations across parliamentary, upazila, union, municipality and city corporation elections.

Boundary: Administrative operations, compliance, personnel, budgets, center readiness, incident records and source-attributed results. No voter persuasion targeting or political-preference profiling.

Document control: Architecture baseline; adapt to current election law and obtain independent legal/security review before production.

Contents and delivery contract

Each numbered section is a development handoff. Deliver features with database migration, backend authorization, frontend permission guard, audit, tests and release notes. No section is production-complete until acceptance tests pass.

No.	Section
1	Scope, personas and navigation
2	Design system and application shell
3	Complete route and screen catalog
4	Screen-by-screen implementation specification
5	Data table and form interaction rules
6	Dynamic menu, routing and permission enforcement
7	Angular project organization
8	API integration mapping
9	Frontend acceptance and test plan
10	System invariants
11	Cross-team contracts

1. Scope, personas and navigation

Primary personas: platform operator, tenant admin, candidate, election manager, area coordinator, center coordinator, finance maker/checker, incident reviewer, results verifier and auditor.

Persona	Landing	Access boundaries
Tenant admin	Tenant dashboard	Tenant-wide settings, scoped election list
Election manager	Election workspace	Assigned elections and areas
Finance officer	Budget workspace	Maker/checker duties separated
Polling agent	Center workspace	Assigned center and shift only
Auditor	Reports and audit	Read-only, masked personal data

2. Design system and application shell

• Angular standalone components, strict TypeScript, reactive forms, signals or RxJS for state; Angular Material for accessible primitives; Tailwind optional for layout.

• Layout: tenant selector, election selector, language toggle, breadcrumb, collapsible menu, notification drawer, profile and MFA settings.

• Components: EemsDataGrid, AreaTreePicker, ElectionSelector, FileUploader, ApprovalTimeline, StatusChip, MoneyInput, DatePicker, AuditDrawer, EmptyState, ConfirmDialog.

• Accessibility: keyboard navigation, visible focus, WCAG 2.2 AA target, Bangla font testing, responsive 320px to desktop.

• Never cache authorization decisions indefinitely; refresh navigation after role or tenant changes.

Token	Specification
Primary	Navy #17395B
Accent	Teal #18749A
Spacing	4px base scale
Typography	14-16px body, 24-32px heading
States	loading, empty, error, permission denied, offline, success

3. Complete route and screen catalog

Module	Routes / screens
Platform	/platform/tenants; /platform/tenants/new; /platform/plans; /platform/audit
Identity	/settings/users; /settings/roles; /settings/permissions; /settings/scopes; /settings/sessions
Elections	/elections; /elections/new; /elections/:id; /elections/:id/schedule; /elections/:id/positions
Candidates	/candidates; /candidates/new; /candidates/:id; /candidate-elections
Geography	/geography/areas; /geography/constituencies; /geography/imports
Polling	/polling/centers; /polling/centers/:id; /polling/booths; /polling/assignments
Teams	/teams; /teams/:id; /members; /assignments; /training; /attendance
Events	/events/calendar; /events/new; /events/:id; /events/permits; /tasks/board
Finance	/finance/budgets; /finance/expenses; /finance/approvals; /finance/vendors; /finance/payments
Inventory	/inventory/items; /inventory/receipts; /inventory/transfers; /inventory/allocations
Operations	/operations/readiness; /operations/agents; /operations/incidents; /operations/timeline
Results	/results/submissions; /results/review; /results/official; /results/reconciliation
Reports	/reports/overview; /reports/finance; /reports/centers; /reports/incidents; /reports/results; /audit

4. Screen-by-screen implementation specification

Screen family	Fields / components	Actions and validation
Election form	name, type, date, scope area, status, milestones	type required; date valid; area compatible; save draft; publish with permission
Candidate form	name, election, position, area, nomination reference, documents	unique mapping; position matches type; no cross-tenant references
Center directory	center code/name, area, address, booth count, verification	filter, import, deduplicate, verify; no unsupported official claims
Team form	team name, leader, election, area, members, dates	scope check, no conflicting role assignment
Event form	title, venue, dates, permit docs, tasks, status	start before end; attach evidence; status transitions
Expense form	budget line, date, vendor, amount, invoice, attachment	amount positive, currency, maker/checker, duplicates
Incident form	center, time, category, narrative, evidence, reviewer	source, timestamp, confidentiality, immutable review history
Result entry	election, center, position, candidate counts, source document	nonnegative totals, source type, independent verifier
Role editor	permission matrix, area/election scopes, version	prevent privilege escalation, require admin approval
Report filters	tenant-scoped election, area, dates, status	server-side filters, masked export, audit exports

5. Data table and form interaction rules

• Every list: filter, sort, page, export (where authorized), skeleton loading, empty state, error retry and selected election context.

• Every form: create/edit/read-only modes; dirty-form warning; inline field errors; server validation mapping; optimistic concurrency token.

• Area selector: only selectable descendants of user authorized area scope; display breadcrumbs and hierarchy.

• Upload: max size and MIME from server configuration, client preview, progress, checksum, scan-pending state.

• Approval timeline: actor, time, decision, comment; never silently overwrite approved financial records.

• Offline mode: drafts only for explicitly supported workflows, encrypted storage, user confirmation before sync; do not persist access tokens in localStorage.

6. Dynamic menu, routing and permission enforcement

```
export interface MenuItem {
  id: string; title: string; icon?: string; route?: string;
  permission?: string; children: MenuItem[];
}
export const routes: Routes = [
  { path: 'elections', canMatch: [authGuard, permissionGuard],
    data: { permission: 'election.read' },
    loadChildren: () => import('./features/elections/election.routes')
      .then(m => m.ELECTION_ROUTES) },
];
```

• GET /api/v1/navigation/my-menu returns only authorized menu items, but API always re-checks rights.

• Permissions: election.read/create/update, finance.expense.create/approve, polling.center.read, results.submit/verify, audit.read.

• Permission guards check tenant and election context; forbidden routes show 403 page, not an infinite redirect.

• Use a single API client with typed DTOs, retry only idempotent reads, cancellation, correlation IDs and ProblemDetails handling.

7. Angular project organization

```
src/app/
  core/{auth,api,guards,interceptors,models}/
  layout/{shell,sidebar,topbar}/
  shared/{components,forms,directives,pipes}/
  features/{tenants,identity,elections,candidates,geography,polling,
            teams,events,finance,inventory,operations,results,reports}/
  app.routes.ts
  app.config.ts
```

• Each feature: pages/, components/, data-access/, models/, feature.routes.ts, feature.spec.ts.

• Generate OpenAPI TypeScript client during CI; never hand-copy backend DTOs into frontend.

• Prefer route-level lazy loading and typed Reactive Forms.

8. API integration mapping

Frontend feature	Read	Write
Election list/form	GET /elections, /elections/{id}	POST /elections; PUT /elections/{id}
Area tree	GET /areas/tree	POST /areas (platform-only)
Polling centers	GET /polling-centers	POST /polling-centers
Expenses	GET /expenses	POST /expenses; POST /expenses/{id}/submit
Approval queue	GET /approvals/pending	POST /expenses/{id}/approve
Results	GET /results/submissions	POST /results/submissions; POST /results/{id}/verify
Menu	GET /navigation/my-menu	none

9. Frontend acceptance and test plan

• Unit: validators, area selector, permission directive, table filters and date formatting.

• Component: form error messages, loading/empty/error states, approval confirmation and document upload.

• E2E: tenant switching, unauthorized route, candidate creation, expense maker-checker, result discrepancy workflow.

• Accessibility: keyboard-only, screen reader landmarks, 200% zoom, Bangla typography, color contrast.

• Performance: route chunks, virtualized long tables, no unbounded center data fetch.

• Definition of done: UX approved, typed API integrated, permission checks tested, audit events observable, docs updated.

10. System invariants

- Every tenant-owned row has tenant_id; all APIs derive tenant from authenticated context, never client input.
- An election has a type, schedule, geographic scope and lifecycle state. Position must belong to election type.
- Permissions are enforced in API and data access; hiding a menu is never authorization.
- Official results are clearly distinguished from unverified field submissions.
- Personal data is minimized; do not store inferred political preference or voter-level persuasion scores.

11. Cross-team contracts

Concern	Contract
Identifiers	UUID for tenant/election/candidate; bigint for canonical geographic codes where appropriate
Date/time	ISO 8601 UTC timestamps; election dates in local election timezone
API	/api/v1; RFC 7807 ProblemDetails; cursor/page pagination
State	Draft -> Submitted -> Reviewed -> Approved/Returned; record actor and time
Files	Object storage, malware scanning, SHA-256, signed downloads
Localization	Bangla/English display strings, stable English permission codes